Миграции и применение схемы

Механизм

backend/app/schema.py содержит три уровня:

Функция	Что делает
reconcile(engine)	добавляет недостающие колонки (аддитивная сверка)
apply_migrations(engine)	применяет упорядоченный реестр миграций, записывая версии в schema_migrations
ensure_schema(engine)	вызывает оба шага; безопасно вызывать на каждом старте

Дополнительно в репозитории лежат SQL-файлы (db/schema.sql, backend/migrations/001_schema.up.sql, 002_seed.sql) — они описывают базовую схему и начальное наполнение.

Правила

1. **Миграция только вперёд.** Откат — восстановление из копии.
2. **Идемпотентность.** Повторный старт не должен ничего менять; это проверяется запуском api дважды подряд в CI (portability.yml, job upgrade-path).
3. **Аддитивность предпочтительна.** reconcile умеет добавлять колонки; удаление и переименование требуют явной миграции с обоснованием.
4. **Сид данных отделён от схемы** (002_seed.sql, AEZA_SEED_DEMO_DATA): демо-данные не должны быть обязательным следствием миграции.
5. **Никаких «ручных правок на проде».** Если нужно — это миграция в репозитории.

Обновление установленного экземпляра

1. make backup.
2. Остановить стек, обновить образы до зафиксированных версий.
3. Поднять: приложение применит недостающие шаги схемы.
4. make smoke-test, затем проверка нескольких расчётов и снимка.

Проверка

Утверждение	Команда	CI
Схема применяется на пустой базе	make compose-up + make smoke-test	portability.yml → compose-smoke
Повторный старт безопасен	два запуска api подряд	portability.yml → upgrade-path
Копия восстанавливается	make backup → make restore → make smoke-test	portability.yml → backup-restore